Manual de creación de la máquina virtual

Este manual describe, paso a paso, cómo se creó desde cero una máquina virtual con **Debian 13** sobre **KVM** y cómo se desplegó en ella **WordPress** servido por **Nginx**, **PHP-FPM** y **MariaDB**. La aplicación se ejecuta en contenedores gestionados con **Podman**, de modo que la misma definición pueda reutilizarse después en la instancia de AWS.

1. Selección del entorno de virtualización

Se eligió **KVM** (*Kernel-based Virtual Machine*) con **QEMU** y **libvirt**, administrado con **virt-manager** y la herramienta de línea de comandos **virt-install**. La actividad recomienda VirtualBox por su sencillez, pero KVM se consideró más adecuado por los siguientes motivos:

- **Hipervisor integrado en el núcleo de Linux:** convierte al propio sistema anfitrión en hipervisor y aprovecha directamente las extensiones de virtualización del procesador (Intel VT-x o AMD-V), con un rendimiento cercano al nativo.
- **Dispositivos paravirtualizados (virtio)** para disco y red, que reducen la sobrecarga de la emulación.
- **Software libre** incluido en los repositorios de Debian, sin licencias ni módulos externos que compilar.
- **Relación directa con la nube:** el hipervisor Nitro de las instancias EC2 de AWS está basado en la tecnología de KVM (Amazon Web Services, s. f.), lo que facilita la comparación posterior entre ambos entornos.

Configuración del anfitrión. Antes de crear la máquina se comprobó que el procesador admite virtualización y se instalaron los paquetes necesarios:

```
# Debe devolver un número mayor que 0
egrep -c '(vmx|svm)' /proc/cpuinfo


sudo apt update
sudo apt install -y qemu-system-x86 libvirt-daemon-system libvirt-clients
virtinst virt-manager
sudo usermod -aG libvirt,kvm $USER        # cerrar sesión y volver a entrar

# Red NAT predeterminada de libvirt (virbr0, 192.168.122.0/24)
sudo virsh net-start default
sudo virsh net-autostart default
```

2. Creación de la máquina virtual desde cero

Tabla 1. Características de la máquina virtual.

Parámetro	Valor
Nombre / hostname	debian13-actividad2
Sistema operativo	Debian 13 «trixie», 64 bits (imagen netinst)
CPU y memoria	2 vCPU y 4 GB de RAM
Disco	25 GB, formato qcow2 con bus virtio (asignación dinámica)
Red	Red NAT «default» de libvirt, tarjeta virtio
Usuario	miguelguzman (con permisos de sudo)

Pasos realizados:

1. Descargar la imagen **netinst** de Debian 13 desde debian.org y verificar su integridad con el archivo de sumas de comprobación: `sha512sum -c SHA512SUMS --ignore-missing`.
2. Crear la máquina con virt-manager (Archivo → Nueva máquina virtual → Medio de instalación local) o con el siguiente comando equivalente:

```
virt-install --name debian13-actividad2 \
  --memory 4096 --vcpus 2 \
  --disk size=25,format=qcow2,bus=virtio \
  --cdrom ~/ISOs/debian-13-amd64-netinst.iso \
  --osinfo debian13 \
  --network network=default,model=virtio \
  --graphics spice
```

1. En el instalador de Debian: idioma español, distribución de teclado, **hostname** `debian13-actividad2`, contraseña de root y usuario `miguelguzman`.
2. Particionado **guiado, utilizando todo el disco**, con todos los ficheros en una partición.
3. En la selección de software se marcaron solo **«Servidor SSH»** y **«Utilidades estándar del sistema»**, sin entorno de escritorio, para reducir el consumo de recursos y la superficie de ataque.
4. Instalar el cargador de arranque **GRUB** en el disco virtual, reiniciar y retirar la ISO.

5. Configuración posterior: actualizar el sistema, añadir el usuario al grupo `sudo`, instalar el agente invitado de QEMU y obtener la IP de la máquina para conectarse por SSH desde el anfitrión:

```
su -c "apt update && apt full-upgrade -y && apt install -y sudo qemu-guest-agent"
su -c "usermod -aG sudo miguelguzman"     # cerrar sesión y volver a entrar
ip -4 addr show                           # p. ej. 192.168.122.X

# Desde el anfitrión
ssh miguelguzman@192.168.122.X
```

3. Instalación de la aplicación web

Se seleccionó **WordPress**, una de las aplicaciones propuestas. En lugar de Apache se utilizó **Nginx** con **PHP-FPM**, una combinación muy extendida en producción, y **MariaDB** como base de datos. Cada componente se ejecuta en su propio contenedor con **Podman en modo sin privilegios** (*rootless*): no requiere un servicio en segundo plano con permisos de administrador y está disponible en los repositorios oficiales de Debian 13.

Tabla 2. Contenedores de la aplicación.

Contenedor	Imagen	Función
wp-nginx	localhost/wp-nginx:1.0 (basada en nginx:1.28-alpine)	Recibe las peticiones HTTP en el puerto 8080, sirve los archivos estáticos y envía los .php a PHP-FPM.
wp-app	localhost/wp-app:1.0 (basada en wordpress:6.8-php8.3-fpm)	Ejecuta WordPress con PHP-FPM; límites de subida ampliados a 64 MB.
wp-db	mariadb:11.4	Base de datos. Solo accesible desde la red interna de los contenedores.

Los datos se guardan en dos volúmenes persistentes: `wordpress_db_data` (base de datos) y `wordpress_wp_data` (archivos de WordPress). Los archivos del despliegue son `compose.yaml`, `.env` con las credenciales, y dos *Dockerfile* que añaden la configuración de Nginx y los límites de PHP a las imágenes oficiales.

Pasos realizados:

1. Instalar Podman y sus herramientas: `sudo apt install -y podman podman-compose git`.
2. Permitir que los contenedores del usuario sigan activos sin sesión iniciada y arranquen con el sistema: `sudo loginctl enable-linger miguelguzman` y `systemctl --user enable --now podman-restart.service`.

3. Copiar los archivos del despliegue a la máquina (`git clone` del repositorio del grupo) y crear el archivo de credenciales a partir de la plantilla: `cp .env.example .env`, cambiando las contraseñas.
4. Construir las imágenes propias y levantar los servicios:

```
cd ~/wordpress-podman
podman-compose build        # construye localhost/wp-app y localhost/wp-nginx
podman-compose up -d        # crea la red, los volúmenes y los 3 contenedores
podman ps                   # wp-db debe aparecer como (healthy)
```

4. Despliegue y prueba

Una vez en marcha los contenedores, se abrió en el navegador del anfitrión la dirección `http://192.168.122.X:8080` y se completó el asistente de instalación de WordPress (idioma, título del sitio y usuario administrador). Después se realizaron las siguientes pruebas:

Tabla 3. Pruebas de funcionamiento.

Prueba	Cómo se comprueba	Resultado esperado
Servicio activo	`curl -I http://localhost:8080` dentro de la VM	HTTP 200 o 302 y cabecera `Server: nginx`
Acceso desde el anfitrión	Navegador en `http://192.168.122.X:8080`	Página de WordPress
Base de datos	Publicar una entrada de prueba	La entrada aparece en la portada
Subida de archivos	Subir una imagen de más de 2 MB a la biblioteca de medios	Se sube sin error (límite de 64 MB)
Enlaces permanentes	Ajustes → Enlaces permanentes → «Nombre de la entrada»	Las URL amigables abren la entrada
Seguridad básica	`curl -I http://localhost:8080/.htaccess`	HTTP 403 (archivos ocultos bloqueados)
Persistencia	Reiniciar la VM con `sudo reboot`	Los contenedores arrancan solos y los datos se conservan

5. Problemas encontrados

- **Nombres cortos de imagen:** Podman no asume Docker Hub como registro predeterminado y pedía elegir uno. Se resolvió usando nombres completos, por ejemplo `docker.io/library/mariadb:11.4`.
- **Puertos privilegiados:** en modo sin privilegios Podman no puede abrir puertos inferiores a 1024, por lo que el 80 de Nginx se publica en el **8080** del anfitrión.

- **Límite de subida de PHP:** Nginx aceptaba archivos de 64 MB, pero PHP mantenía su límite predeterminado de 2 MB y las subidas grandes fallaban. Se creó un *Dockerfile* que añade un `uploads.ini` con límites de 64 MB.
- **Orden de arranque:** si Nginx arranca antes que PHP-FPM no encuentra el servicio `wordpress`. Se añadió una comprobación de salud a MariaDB y dependencias entre servicios en `compose.yaml`; si ocurre, basta con `podman restart wp-nginx`.
- **Arranque tras reinicio:** la política `restart` no basta en modo sin privilegios; fue necesario habilitar *linger* y el servicio `podman-restart`.
- **Identificador del sistema operativo en virt-install:** si la versión de libosinfo del anfitrión no reconoce `debian13`, se puede usar `--osinfo detect=on,require=off`.

6. Vídeo de demostración de la máquina virtual

URL del vídeo: pendiente de añadir

El vídeo muestra la máquina en virt-manager, el arranque de Debian, la ejecución de `podman ps`, el acceso a WordPress desde el navegador del anfitrión, la publicación de una entrada y la comprobación de que el sitio sigue disponible tras reiniciar la VM.

Referencias

Amazon Web Services. (s. f.). *AWS Nitro System*. Recuperado el 5 de noviembre de 2026, de https://aws.amazon.com/ec2/nitro/

Debian. (s. f.). *Guía de instalación de Debian GNU/Linux*. Recuperado el 5 de noviembre de 2026, de https://www.debian.org/releases/stable/installmanual

Libvirt. (s. f.). *virt-install*. Recuperado el 5 de noviembre de 2026, de https://libvirt.org/